Examen práctico — Práctica 1 (Propuesta 2): Búsqueda por rango de años

Duración máxima: 1 h 45 min. Sin Internet ni asistentes de IA.

Dispones de:

- Tu implementación de la práctica 1.
- Las 16 pruebas completas de la práctica.
- Las 10 pruebas del grupo **X** correspondientes a este examen.
- Seminario de C++ en https://www.dlsi.ua.es/asignaturas/eda/seminario-c++-parte1.pdf
- Copia del manual de referencia de C++ en https://www.dlsi.ua.es/asignaturas/eda/cpp

Ejecuta `./run_tests.sh X` para ejecutar las pruebas del examen y ver tu calificación.

Ampliación: consultas por rango de años

Los catálogos crecen y las consultas por identificador o por etiqueta ya no bastan: hace falta poder preguntar qué recursos hay entre dos años, opcionalmente restringiendo la búsqueda a una etiqueta.

Todos los métodos de esta ampliación son **añadidos**: no puedes cambiar ninguna firma de la práctica (los grupos A, B y C deben seguir pasando). Recuerda declarar los métodos nuevos en `Resource.h` y `Catalog.h`.

1. Método `Resource::compareByYear`

```
int compareByYear(const Resource& other) const;
```

Comparador de tres vías que se usará para ordenar los resultados:

- Devuelve **exactamente** `-1`, `0` o `1`.
  - `-1` si este recurso va **antes** que `other`.
  - `1` si va **después**.
  - `0` si son equivalentes según el criterio de orden.
- Criterio: `year` **ascendente** y, a igualdad de año, `id` **ascendente** (orden lexicográfico).
- Devuelve `0` únicamente si coinciden el año **y** el `id`.
- El método es `const` y no modifica ninguno de los dos recursos.

  Atención: no vale devolver `year - other.year`. Los valores devueltos deben ser `-1`, `0` o `1`, y el desempate por `id` es obligatorio.

2. Método `Catalog::findByYearRange`

```
vector<Resource> findByYearRange(int from, int to, const string&
tag) const;
```

- Devuelve los recursos cuyo año cumple `from <= year <= to` y que **además** contienen la etiqueta `tag`.
- Si `tag` es la **cadena vacía**, no se filtra por etiqueta: se devuelven todos los recursos del rango.
- Las reglas para comparar dos etiquetas son las de `Resource::hasTag` (normaliza a minúsculas, **sin** recortar espacios).
- El resultado se ordena usando `Resource::compareByYear` (año ascendente y, a igualdad de año, `id` ascendente).
- Si `from > to`: devuelve un vector vacío, aunque haya recursos con esa etiqueta.
- El catálogo **no se modifica** (el método es `const`).

Casos que debes tener en cuenta: rango de un solo año (`from == to`), catálogo vacío, etiqueta inexistente, etiqueta escrita en mayúsculas.

No se introducirán años negativos.

3. Comando `YEARS <from> <to> <tag|->`

- `from` y `to` son los extremos del rango, ambos incluidos.
- El tercer argumento es una etiqueta, o `-` si no se quiere filtrar por etiqueta (igual que en el comando `ADD`).
- Imprime `COUNT <k>` seguido de `k` líneas con los recursos encontrados (formato de `operator<<`), en el orden indicado.
- Si faltan argumentos o si `from` o `to` no son enteros válidos: `INVALIDCMD`.

Ejemplo:

```
LOAD data/sample_ok.txt
OK 6
YEARS 1994 2011 -
COUNT 4
bk04 (1994) The Pragmatic Programmer [craft,software]
bk01 (1999) Clean Code [craft,software]
bk02 (2008) Introduction to Algorithms [algorithms,cs]
bk05 (2011) Structure and Interpretation [cs,lisp,programming]
YEARS 1994 2011 CRAFT
COUNT 2
bk04 (1994) The Pragmatic Programmer [craft,software]
bk01 (1999) Clean Code [craft,software]
YEARS 2011 1994 -
COUNT 0
YEARS 2011 x -
INVALIDCMD
```

(`bk03` y `bk06` quedan fuera del rango; en la segunda consulta sólo `bk01` y `bk04` tienen la etiqueta `craft`, y la búsqueda funciona igual escrita en mayúsculas.)

Evaluación

- Nota = número de pruebas pasadas.
- El código debe compilar sin errores ni warnings.
- No se puede modificar la carpeta `tests/`.